

Android Mobile App Penetration Testing – Start to Finish (With Physical Device)

1. Prerequisites

- A **Windows PC**
- A **physical Android device** (rooted = more powerful testing)
- The **APK** you want to test (from Play Store or extracted)
- An internet connection
- Some basic technical knowledge (command line, Android basics)

□ Android

1. APK file (latest build)
2. Pinned and unpinned certificate versions (if applicable)
3. Test credentials and user roles (*for GreyBox testing*)
4. Walkthrough of application functionality (*for GreyBox testing*)

□ iOS

1. IPA file (latest build)
2. Installation method:
 - To Bind UDID of the iOS test device
 - TestFlight access (*with test email/invite*)
3. Test credentials and user roles (*if applicable*)
4. Walkthrough of application functionality (*for GreyBox testing*)

2. Set Up Your Windows Environment

Install the following tools:

Tool	Purpose	How to Get
ADB	Connect to Android device	Install via Android Platform Tools from Google
JDK	Required for tools like APKTool	Download JDK
APKTool	Decompile APKs	https://ibotpeaches.github.io/Apktool/
JADX	View app source code	https://github.com/skylot/jadx
Burp Suite	Intercept network traffic	https://portswigger.net/burp
Frida + Objection	Bypass SSL, root detection, runtime hooks	Use WSL or install via Python (details below)
MobSF (optional)	Automated analysis	Install in WSL2 (Ubuntu) or use the MobSF Docker image

3. Prepare Your Physical Android Device

A. Enable Developer Mode

- Settings → About Phone → tap Build Number 7 times
- Go back → Developer Options → Enable:
 - USB Debugging
 - (If rooted) Root Access

B. Connect via USB

- Connect to Windows with cable
- Run: Command

```
adb devices
```

You should see your device listed.

4. Get the APK File

A. Download

- from Play Store convert into apk other service use website https://apps.evozi.com/apk-downloader/#google_vignette
- Use APKPure, APKCombo, or Aurora Store

B. Extract app from Android device into the system

cmd

```
adb shell pm list packages
```

```
adb shell pm path com.example.targetapp
```

```
adb pull /data/app/xxxxx/base.apk
```

5. Perform Static Analysis

A. Use JADX:

- Open jadx-gui
 - Load APK → Review Java code
 - Look for:
 - API URLs
 - Tokens or keys Bearer Authorization apikey token= secret=
 - Sensitive logic
 - Debug info
 - Debug
 - isDebuggerConnected
 - waitForDebugger
 - Xposed
 - hook
 - reflection
 - Frida
 - Often found in `Constants.java`, `Config.java`, or `BuildConfig.java`
- Sensitive Logic or Business Rules
- License checks
 - Payment validation

- Root/jailbreak detection
- Feature flags
- Debug bypass logic
- Crypto Misuse
 - Cipher
 - MessageDigest
 - KeyGenerator

`apksigner verify --print-certs C:\Users\YourName\Desktop\MyApp.apk`

`certutil -hashfile C:\Users\YourName\Desktop\MyApp.apk SHA256`

B. Use APKTool to Decompile:

❶ Open Command Prompt (CMD)

❷ Run APKTool Decompile Command:

`apktool d app.apk -o output_folder`

Example:

`apktool d Facebook.apk -o Facebook_src`

- d = decompile
- -o = output folder name

✅ Output: The app's source will be extracted into Facebook_src folder.

Inspect:

- AndroidManifest.xml for exported components
- Look for misconfigurations

`<activity android:exported="true" ... />`

`<service android:exported="true" ... />`

`<receiver android:exported="true" ... />`

`<provider android:exported="true" ... />`

- Check if **exported=true** without permission → ❌ Potential component exposure
- Make sure sensitive components are **not publicly accessible**

Key Focus Areas in Static Analysis

□ Manifest/Plist Files

- Misconfigured components (e.g., exported activities/services)
- Insecure permissions
- Debuggable flags enabled

□ Code Review (Decompiled)

- Hardcoded credentials, API keys, tokens
- Insecure cryptographic implementations
- Sensitive information leakage in strings

□ File Storage & Data Handling

- Insecure file storage paths (e.g., external storage)
- Improper use of shared preferences or SQLite

APK tempering/Reverse engineering

APK tampering refers to the process of modifying an Android application package (APK) file, typically to bypass restrictions, remove ads, unlock premium features, or alter the behaviour of the app. APK files are the installation packages for Android apps, and tampering with them usually involves manipulating the code or resources within the APK to achieve specific objectives.

Common Types of APK Tampering:

1. **Removing Ads:** Some users modify APK files to remove ads or advertisements that are part of a free app, making the app more user-friendly or less intrusive.
2. **Bypassing Licensing and Verification:** APK tampering can be used to bypass license verification mechanisms, such as premium or paid features, making the app function as if it were legitimately purchased.
3. **Modifying App Behaviour:** This can include altering functionality, changing how the app interacts with the server, or bypassing certain security measures.
4. **Injecting Malicious Code:** In some cases, malicious actors may tamper with APK files to include malware, spyware, or ransomware.

How APK Tampering is Done:

1. **Decompiling the APK:** The APK is extracted and decompiled to view its code, resources, and structure. Tools like APKTool, JADX, or JDK can be used for decompiling APK files.
2. **Modifying the Code:** Once decompiled, the code can be edited using tools like Android Studio or other code editors. Modifications can include changes to the app's logic, resources, or features.
3. **Recompiling the APK:** After making changes, the APK is recompiled back into a working format, ready to be installed.
4. **Signing the APK:** Since APK files must be signed to be installed on Android devices, the tampered APK will need to be re-signed with a new key. Tools like Keytool or Jarsigner can be used for signing the APK.

APK tampering using APK Tool

APK tampering involves modifying an APK (Android Application Package) file, which is typically done for reverse engineering, debugging, or testing. APKTool is a powerful tool for decompiling APKs, allowing you to inspect, modify, and rebuild them. Here's a step-by-step guide on how to perform APK tampering using APKTool on Windows:

Prerequisites

1. **Install Java:** APKTool requires Java, so make sure Java is installed on your system.
 - Download and install [Java JDK](#).
 - Verify Java installation with the command: `java -version` in Command Prompt.
2. **Install APK Tool:**
 - Download APK Tool from the [official website](#).
 - You need two files:
 - `apktool.jar`
 - `apktool.bat` (Windows batch script)
3. **Set up Environment:**
 - Place `apktool.jar` and `apktool.bat` in a directory like `C:\apktool`.
 - Add the directory to your system PATH to make `apk tool` available globally in the Command Prompt:
 - Right-click **This PC > Properties > Advanced System Settings > Environment Variables**.
 - Under **System Variables**, find **Path**, click **Edit**, and add `C:\apktool` (or wherever you placed the files).

Steps for APK Tampering Using APK Tool

1. *Decompile the APK*

- Open a Command Prompt and navigate to the folder where your APK is stored.
- Use APKTool to decompile the APK

apktool d your-app.apk

```

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.5198]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Aadil rajani\OneDrive - protechmanize.com\mobile testing tools>apktool d DivaApplication.apk
I: Using Apktool 2.10.0 on DivaApplication.apk with 4 thread(s).
I: Baksmaling classes.dex...
I: Loading resource table...
I: Decoding file-resources...
I: Loading resource table from file: C:\Users\Aadil rajani\AppData\Local\apktool\framework\1.apk
I: Decoding values */* XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Regular manifest package...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...

C:\Users\Aadil rajani\OneDrive - protechmanize.com\mobile testing tools>

```

- This will extract the contents of the APK into a folder named `your-app` (or any name you choose). This folder will contain the resources and code in an understandable format (e.g., XML, Smali).

Name	Status	Date modified	Type	Size
lib		05-12-2024 02:46 PM	File folder	
original		05-12-2024 02:46 PM	File folder	
res		05-12-2024 02:46 PM	File folder	
smali		05-12-2024 02:46 PM	File folder	
AndroidManifest.xml		05-12-2024 02:46 PM	Microsoft Edge HTML Document	4 KB
apktool.yml		05-12-2024 02:46 PM	YAML File	1 KB

2. Modify the Decompiled Files

- **Change Resources:** You can modify resources (such as images, layout files, or strings). You'll find these resources in the `res` folder inside the decompiled folder.
 - Example: Modify `strings.xml` or `image.png`.
- **Change Code:** If you need to modify the behavior of the app, you can work with the Smali code (which is a low-level representation of the app's Java bytecode). You can find the Smali code in the `smali` folder inside the decompiled APK directory.
 - Use a Smali editor like **Smali2Java** to convert back to Java if necessary, or edit Smali directly.

3. Rebuild the APK

- Once you've made your modifications, rebuild the APK using the following command

- This will generate a new APK file located in the `dist` folder inside the `your-app` directory.

```
C:\Users\Aadil rajani\OneDrive - protechmanize.com\mobile testing tools>apktool b DIVANEW
I: Using Apktool 2.10.0 with 4 thread(s).
I: Checking whether sources has changed...
I: Smaling smali folder into classes.dex...
I: Checking whether resources has changed...
I: Building resources...
I: Copying libs... (/lib)
I: Building apk file...
I: Copying unknown files/dir...
I: Built apk into: DIVANEW\dist\DivaApplication.apk
```

4. Sign the APK

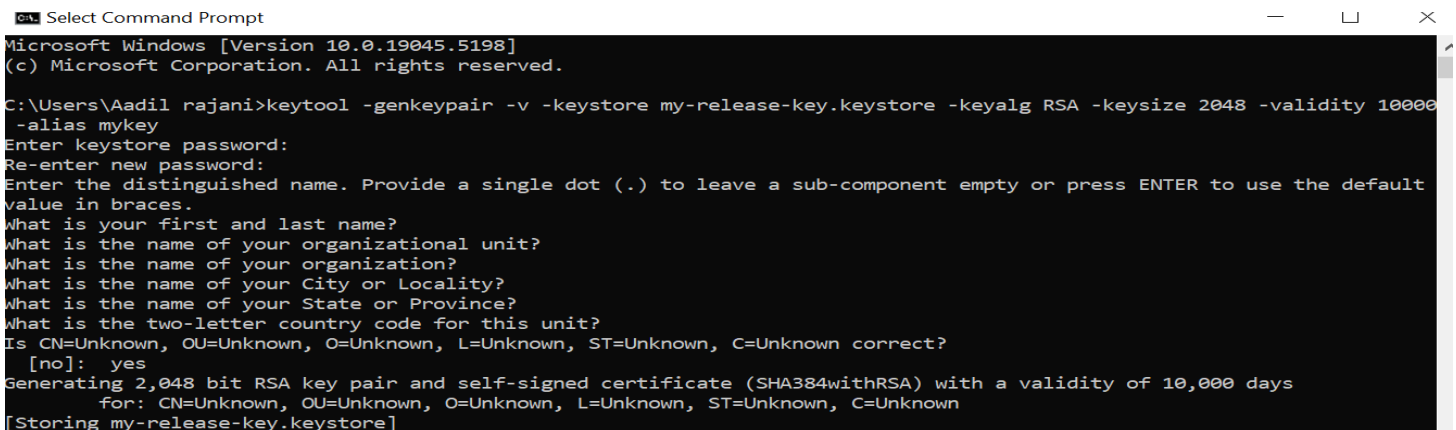
- Since tampered APKs are unsigned, you must sign them before installing them on a device.
- You can use **Java's** `jarsigner` tool or **apksigner** (part of Android SDK) to sign the APK.

Using jarsigner:

- If you don't have a keystore, create one with this command:

```
keytool -genkeypair -v -keystore my-release-key.keystore -keyalg RSA -keysize 2048 -validity 10000 -alias mykey
```

```
jarsigner -verbose -sigalg SHA1withRSA -digestalg SHA1 -keystore my-release-key.keystore dist\your-app.apk mykey
```



```
Microsoft Windows [Version 10.0.19045.5198]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Aadil rajani>keytool -genkeypair -v -keystore my-release-key.keystore -keyalg RSA -keysize 2048 -validity 10000 -alias mykey
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
What is the name of your organizational unit?
What is the name of your organization?
What is the name of your City or Locality?
What is the name of your State or Province?
What is the two-letter country code for this unit?
Is CN=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown correct?
[no]: yes
Generating 2,048 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 10,000 days
for: CN=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
[Storing my-release-key.keystore]
```



```
C:\Users\Aadil rajani>jarsigner -verbose -sigalg SHA1withRSA -digestalg SHA1 -keystore my-release-key.keystore DivaApplication.apk mykey
Enter Passphrase for keystore:
  adding: META-INF/MANIFEST.MF
  adding: META-INF/MYKEY.SF
  adding: META-INF/MYKEY.RSA
signing: AndroidManifest.xml
signing: classes.dex
signing: lib/arm64-v8a/libdivajni.so
signing: lib/armeabi/libdivajni.so
signing: lib/armeabi-v7a/libdivajni.so
signing: lib/mips/libdivajni.so
signing: lib/mips64/libdivajni.so
signing: lib/x86/libdivajni.so
signing: lib/x86_64/libdivajni.so
signing: res/anim/abc_fade_in.xml
signing: res/anim/abc_fade_out.xml
signing: res/anim/abc_grow_fade_in_from_bottom.xml
```

```
signing: res/layout-v21/abc_screen_toolbar.xml
signing: res/layout-v21/content_main.xml
signing: res/menu/menu_main.xml
signing: res/mipmap-hdpi-v4/ic_launcher.png
signing: res/mipmap-mdpi-v4/ic_launcher.png
signing: res/mipmap-xhdpi-v4/ic_launcher.png
signing: res/mipmap-xxhdpi-v4/ic_launcher.png
signing: res/mipmap-xxxhdpi-v4/ic_launcher.png
signing: resources.arsc

>>> Signer
X.509, CN=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
Signature algorithm: SHA384withRSA, 2048-bit key
[trusted certificate]

jar signed.

Warning:
The signer's certificate is self-signed.
The SHA1 algorithm specified for the -digestalg option is considered a security risk and is disabled.
The SHA1withRSA algorithm specified for the -sigalg option is considered a security risk and is disabled.
C:\Users\Aadil rajani>
```

5. Verify the Signature

- To ensure the APK is signed correctly, verify it using the following command

jarsigner -verify -verbose myapp-aligned.apk

```
C:\Users\Aadil rajani>jarsigner -verify -verbose DivaApplication.apk

39898 Thu Dec 05 15:26:16 IST 2024 META-INF/MANIFEST.MF
40019 Thu Dec 05 15:26:16 IST 2024 META-INF/MYKEY.SF
1511 Thu Dec 05 15:26:16 IST 2024 META-INF/MYKEY.RSA
m ? 6540 Thu Dec 05 14:50:48 IST 2024 AndroidManifest.xml
m ? 2182132 Thu Dec 05 14:50:48 IST 2024 classes.dex
m ? 5512 Thu Dec 05 14:50:48 IST 2024 lib/arm64-v8a/libdivajni.so
m ? 13452 Thu Dec 05 14:50:48 IST 2024 lib/armeabi/libdivajni.so
m ? 13460 Thu Dec 05 14:50:48 IST 2024 lib/armeabi-v7a/libdivajni.so
m ? 5488 Thu Dec 05 14:50:48 IST 2024 lib/mips/libdivajni.so
m ? 6048 Thu Dec 05 14:50:48 IST 2024 lib/mips64/libdivajni.so
m ? 5164 Thu Dec 05 14:50:48 IST 2024 lib/x86/libdivajni.so
m ? 5704 Thu Dec 05 14:50:48 IST 2024 lib/x86_64/libdivajni.so
m ? 388 Thu Dec 05 14:50:48 IST 2024 res/anim/abc_fade_in.xml
m ? 388 Thu Dec 05 14:50:48 IST 2024 res/anim/abc_fade_out.xml
m ? 852 Thu Dec 05 14:50:48 IST 2024 res/anim/abc_grow_fade_in_from_bottom.xml
m ? 508 Thu Dec 05 14:50:48 IST 2024 res/anim/abc_popup_enter.xml
m ? 508 Thu Dec 05 14:50:48 IST 2024 res/anim/abc_popup_exit.xml
m ? 852 Thu Dec 05 14:50:48 IST 2024 res/anim/abc_shrink_fade_out_from_bottom.xml
m ? 396 Thu Dec 05 14:50:48 IST 2024 res/anim/abc_slide_in_bottom.xml
```

```

m ? 1956 Thu Dec 05 14:50:52 IST 2024 res/mipmap-mdpi-v4/ic_launcher.png
m ? 4366 Thu Dec 05 14:50:52 IST 2024 res/mipmap-xhdpi-v4/ic_launcher.png
m ? 7007 Thu Dec 05 14:50:52 IST 2024 res/mipmap-xxhdpi-v4/ic_launcher.png
m ? 9490 Thu Dec 05 14:50:52 IST 2024 res/mipmap-xxhdpi-v4/ic_launcher.png
m ? 234112 Thu Dec 05 14:50:52 IST 2024 resources.arsc

s = signature was verified
m = entry is listed in manifest
k = at least one certificate was found in keystore
? = unsigned entry

- Signed by "CN=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown"
  Digest algorithm: SHA1 (disabled)
  Signature algorithm: SHA1withRSA (disabled), 2048-bit key

WARNING: The jar will be treated as unsigned, because it is signed with a weak algorithm that is now disabled by the security property:
jdk.jar.disabledAlgorithms=MD2, MD5, RSA keySize < 1024, DSA keySize < 1024, SHA1 denyAfter 2019-01-01

```

B. Use JADX:

- Open jadx-gui
- Load APK → Review Java code
- Look for:
 - API URLs
 - Tokens or keys Bearer Authorization apikey token= secret=
 - Sensitive logic
 - Debug info
- Often found in Constants.java, Config.java, or BuildConfig.java

Sensitive Logic or Business Rules

- License checks
- Payment validation
- Root/jailbreak detection
- Feature flags
- Debug bypass logic
- Crypto Misuse
 - Cipher
 - MessageDigest
 - KeyGenerator

6. Dynamic Analysis Set Up Network Traffic Interception (Burp Suite)

✓ A. Configure Burp Suite on Windows

1. Open Burp Suite
2. Navigate to:
 - Proxy → Options → Proxy Listeners

3. Ensure a listener is active on:

Interface: 127.0.0.1

Port: 8080

If not, click "Add" to create it.

4. **B. Configure Android Device to Use Burp as Proxy**

1. Connect Android to Same Network (Wi-Fi)

- Make sure your Android and Windows PC are connected to the same Wi-Fi network.

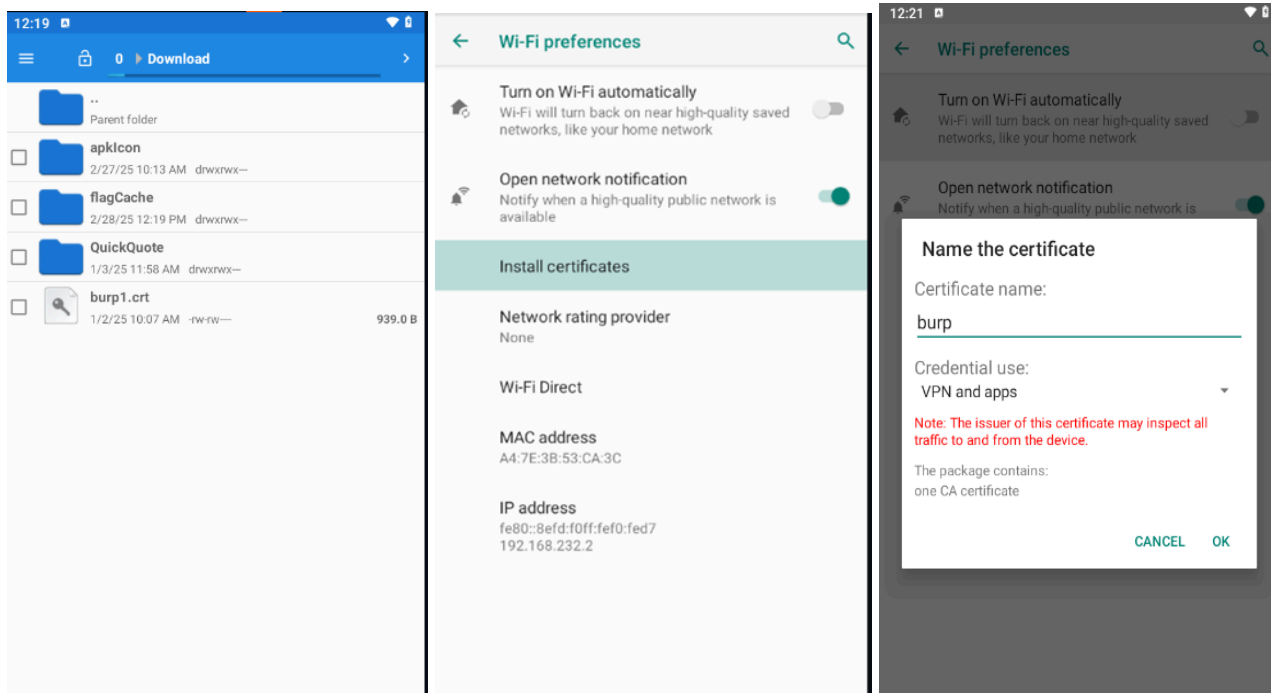
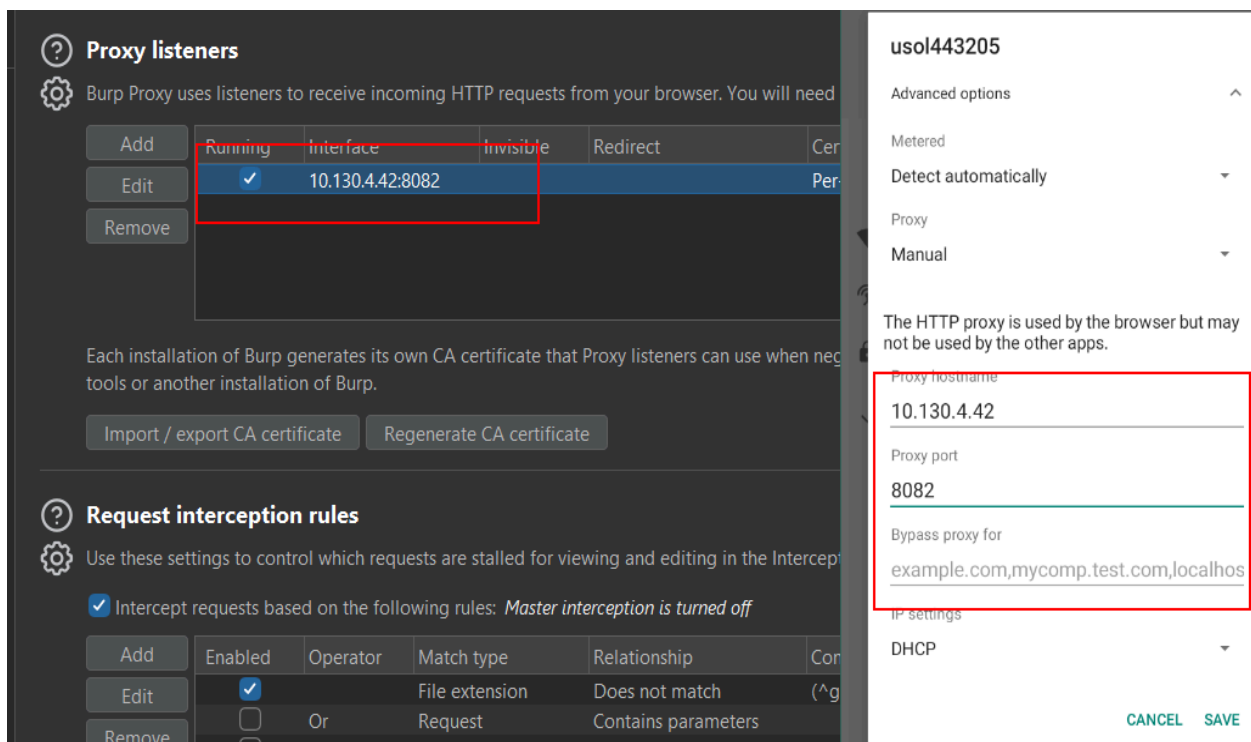
2. Find Your Windows IP Address

- Open Command Prompt on Windows `ipconfig`
- Note the **IPv4 Address** (e.g., 192.168.1.100)

3. Set Proxy on Android

- On Android:
Go to: Settings → Wi-Fi → (Long press your connected network) → Modify network → Advanced
- Set:
 1. **Proxy = Manual**
 2. **Proxy Hostname = Your Windows IP** (e.g., 192.168.1.100)
 3. ****Proxy Port = 8080`**
- Save settings

OR



✓ C. Install Burp Suite CA Certificate on Android

A. Download the Certificate

- On Android browser (while proxy is on), open:

http://burp

- It will download cacert.der file.

B. Rename the Certificate

- Rename the downloaded file from cacert.der to burp.cer

C. Install the Certificate

- Go to:
Settings → Security → Install from storage/SD card
- Select burp.cer to install it as a Trusted Certificate
- And check

The image shows two side-by-side screenshots. The left screenshot is from Burp Suite's 'SSL Scanner' tab, displaying a list of intercepted HTTP requests. The right screenshot shows a mobile browser interface with the YouTube homepage.

Burp Suite SSL Scanner:

Time	Type	Direction	Method	URL
12:23:56 ...	HTTP	→ Request	GET	https://www.google.com/complete/search?client=chrome&gs_ri=chrome-mobile-ext-ans&xs=6&q=4o1t=0&gs_rn=4&gsugkey=...
12:23:56 ...	HTTP	→ Request	POST	https://m.youtube.com/youtubei/v1/log_event?alt=json
12:24:01 ...	HTTP	→ Request	POST	https://m.youtube.com/youtubei/v1/log_event?alt=json
12:24:01 ...	HTTP	→ Request	GET	https://m.youtube.com/

Request Details (Pretty):

```

1 GET /complete/search?client=chrome&gs_ri=chrome-mobile-ext-ans&xs=6&q=4o1t=0&gs_rn=4&gsugkey=
  AIsa5yB10us5sXJ0gBeLFW_DcPcjduqQFQHsqEY HTTP/1.1
2 Host: www.google.com
3 Cookie: NID=
  SCC-B2Hj_Tx_cceme1C1L3FVsyMHZ61uaqy6SftTLrvF3JvYR_t24DymXjeVHVf4Dn0q0XD9W1Luzf9cqcFasAQ0N4cDa8EgFami318A0LupdbJhof0na61V1fC_Fy
  -LRLTAMk1Y0NM3VAKausMDmONSYULGHW20V9uhIMMne780xA54y8agFvzJ IowaFaNvI
4 X-Client-Data: CPs0yGf=
5 Sec-Fetch-Site: none
6 Sec-Fetch-Mode: no-cors
7 Sec-Fetch-Dest: empty
8 User-Agent: Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Mobile Safari/537.36
9 Accept-Encoding: gzip, deflate, br
10 Accept-Language: en-US,en;q=0.9
11 Priority: u=4, i
12 Connection: keep-alive
13
14

```

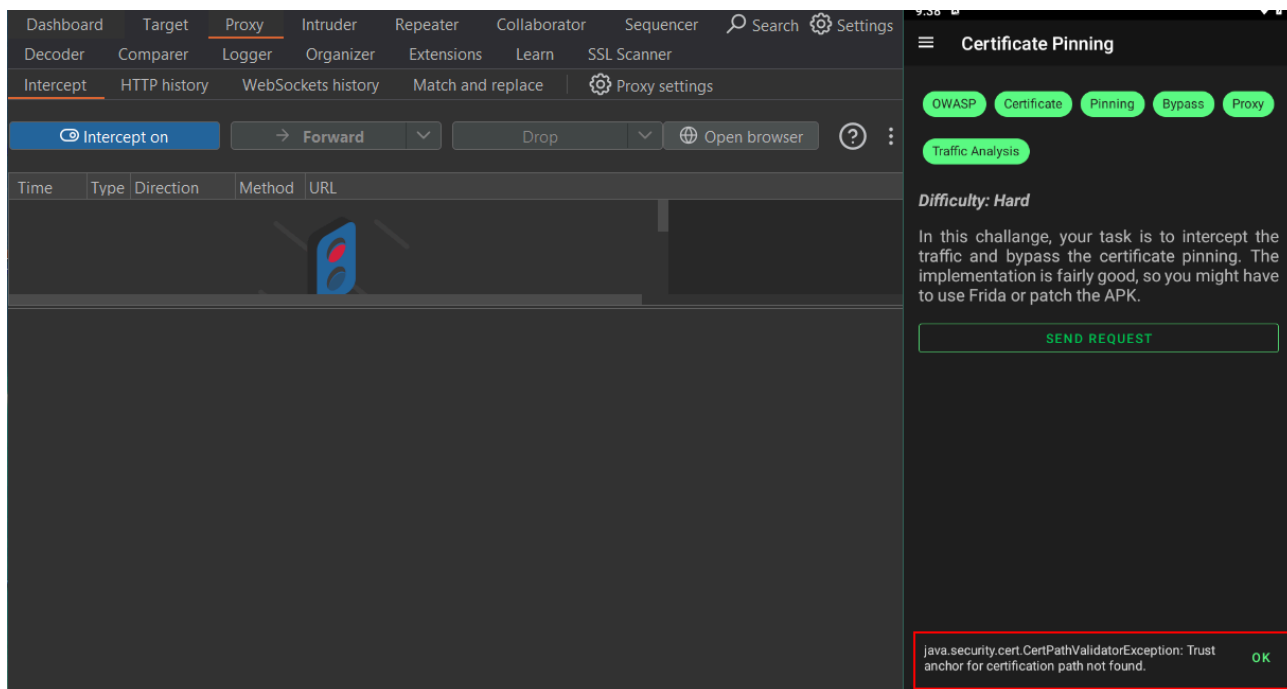
Mobile Browser (m.youtube.com):

The browser shows the YouTube mobile interface with a search bar and a message: "Try searching to get started. Start watching videos to help us build a feed of videos you'll love."

7. Bypass SSL Pinning (If Needed)

pinning bypass refers to a technique used to bypass SSL/TLS certificate validation in mobile or web applications. It involves manipulating or disabling the security mechanisms that check if the server's certificate matches a predefined or "pinned" certificate. This is often done for the purpose of intercepting encrypted traffic during penetration testing or malicious attacks. Bypassing SSL pinning can expose sensitive data and compromise security.

Step1: Connect your application with burp proxy and observe that the request can't be intercepted due to certificate error.



```
pip install frida-tools
```

```
pip install objection
```

A. Push Frida Server to Device

- **Download Frida Server** that matches your device architecture (e.g., frida-server-16.1.4-android-arm64.xz from [Frida GitHub releases](#))
- **Unzip** the file and push it to the device

```
adb push frida-server /data/local/tmp/
```

```
ASUS_Z01QD:/ # adb shell
ASUS_Z01QD:/ # cd /data/local/tmp
ASUS_Z01QD:/data/local/tmp # ./frida
```

- **Set permissions** and run it:

```
adb shell
```

```
$ cd /data/local/tmp/
```

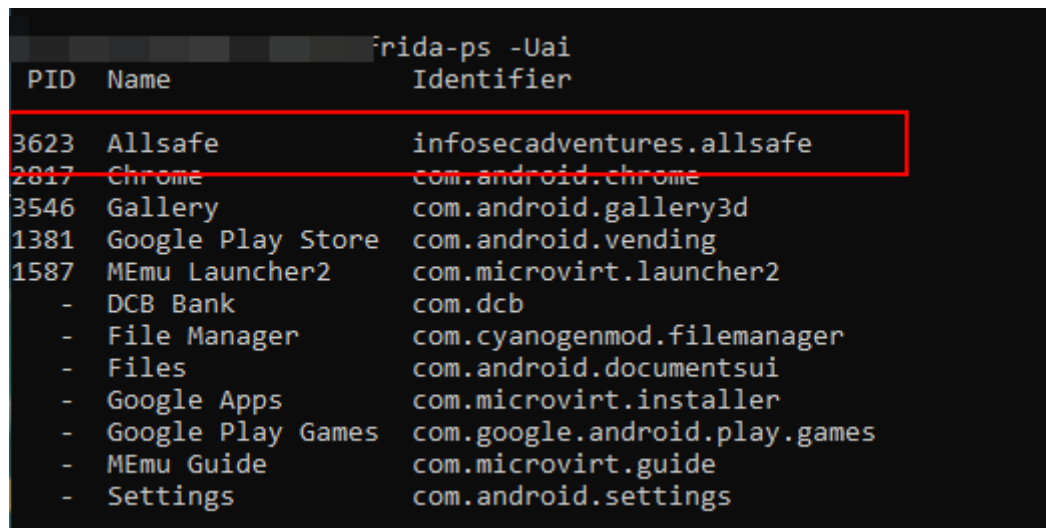
```
$ chmod +x frida-server
```

```
$ ./frida-server & You may need root access to run Frida server properly. For rooted devices:
```

```
$ su
```

B. Find the App's Package Name

```
frida-ps -Uai
```



PID	Name	Identifier
3623	Allsafe	infosecadventures.allsafe
2817	Chrome	com.android.chrome
3546	Gallery	com.android.gallery3d
1381	Google Play Store	com.android.vending
1587	MEmu Launcher2	com.microvirt.launcher2
-	DCB Bank	com.dcb
-	File Manager	com.cyanogenmod.filemanager
-	Files	com.android.documentsui
-	Google Apps	com.microvirt.installer
-	Google Play Games	com.google.android.play.games
-	MEmu Guide	com.microvirt.guide
-	Settings	com.android.settings

This lists all installed apps. Identify your target app (e.g., com.example.app).

C. Attach Objection to the Running App

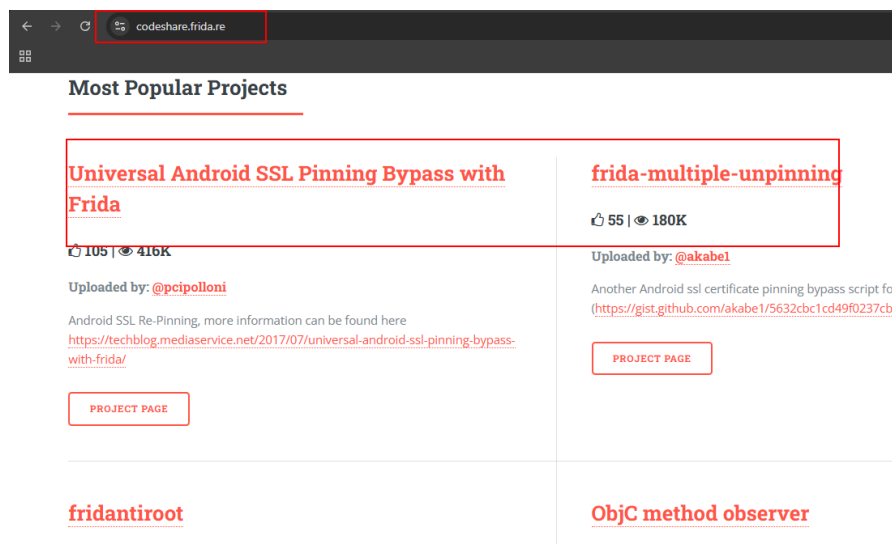
```
objection -g com.example.app explore
```

D. Disable SSL Pinning in Objection

```
android sslpinning disable
```

OR

Go on [Codeshare](#). Frida page for SSL pinning



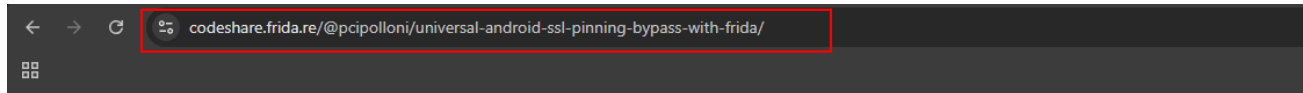
← → ↻ ↻ codeshare.frida.re

Most Popular Projects

Universal Android SSL Pinning Bypass with Frida 👤 105 📄 416K Uploaded by: @pcipolloni Android SSL Re-Pinning, more information can be found here https://techblog.mediaservice.net/2017/07/universal-android-ssl-pinning-bypass-with-frida/ PROJECT PAGE	frida-multiple-unpinning 👤 55 📄 180K Uploaded by: @akabei Another Android ssl certificate pinning bypass script for (https://gist.github.com/akabei/5632cb1cd49f0237cbd) PROJECT PAGE
fridantiroot	ObjC method observer

script.

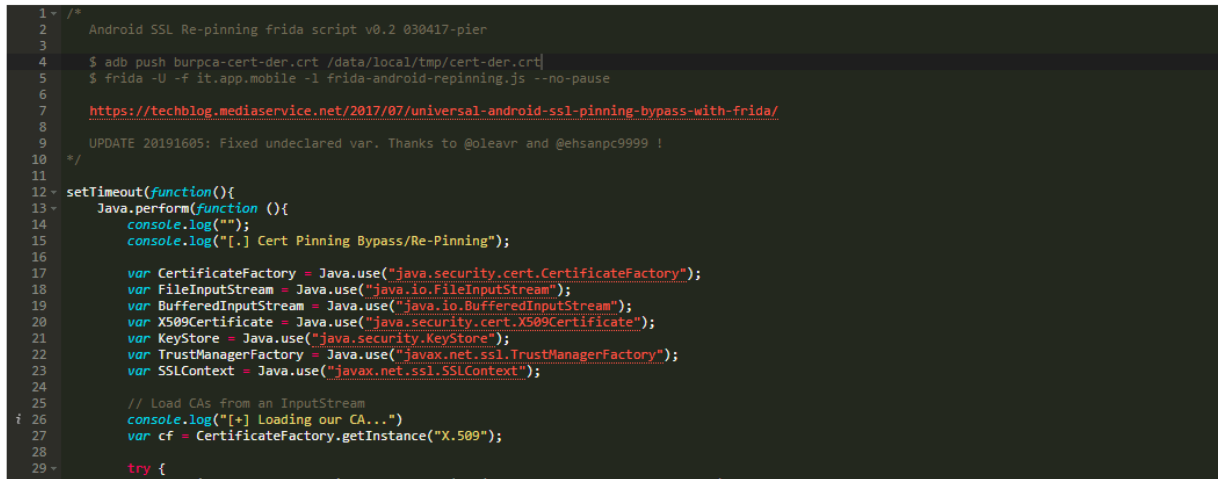
Copy the command to use the following Frida script.



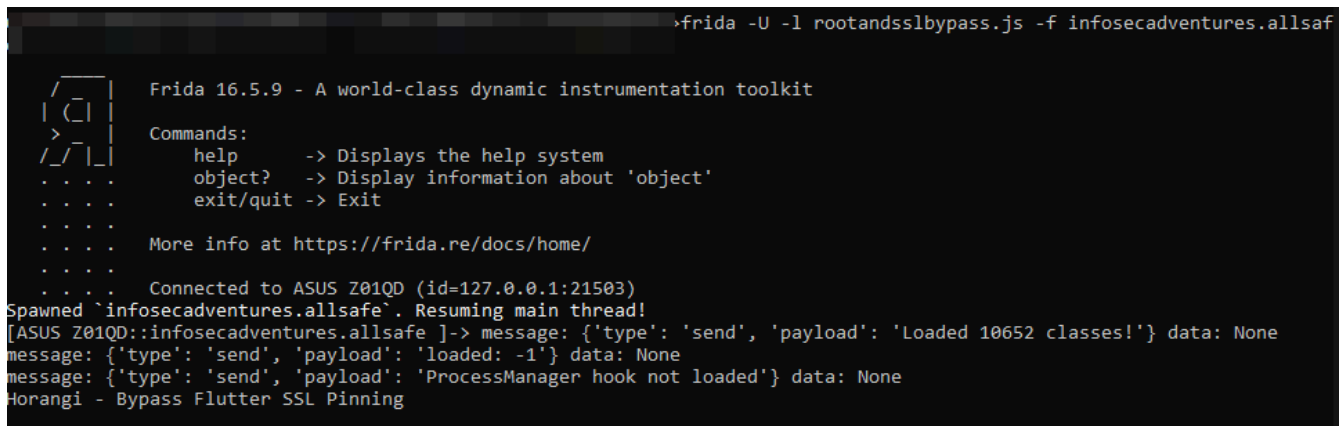
Project: Universal Android SSL Pinning Bypass with Frida

Try this code out now by running

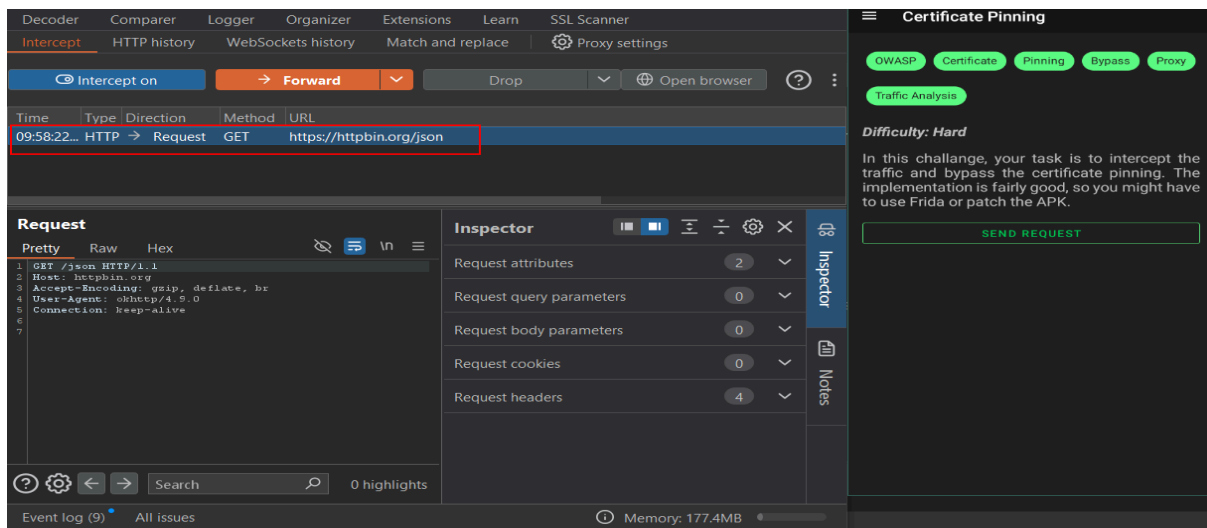
```
$ frida --codeshare pcipolloni/universal-android-ssl-pinning-bypass-with-frida -f YOUR_BINARY
```



Paste the Frida script and run with your package name .



SSL certificate is successfully bypass as we can see request can be intercepted using burp suite

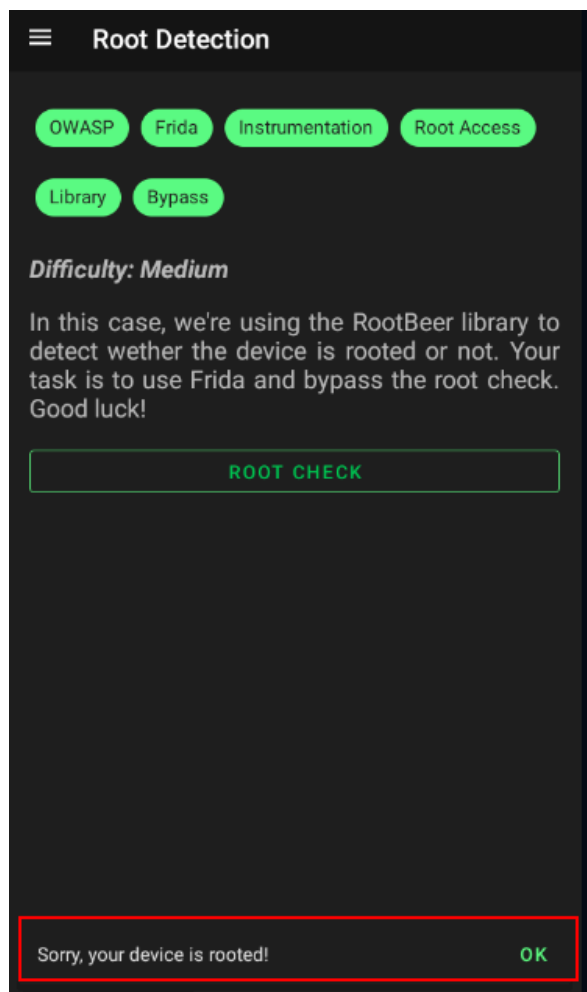


Root Detection Bypass

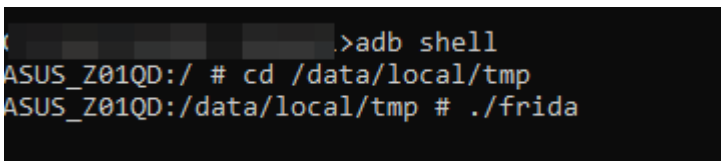
Root Detection Bypass Detection refers to methods or techniques used to detect whether a device (usually a smartphone or computer) has been "rooted" (on Android) or "jailbroken" (on iOS). Rooting or jailbreaking allows a user to gain superuser access to the operating system, often enabling them to remove restrictions set by the manufacturer or service provider.

However, apps, software, or systems that are designed to protect against such modifications (like security apps or financial apps) often implement **root detection** to prevent unauthorized access or manipulation. Root detection helps detect if the device has been tampered with, which might compromise security.

Step1: Observe that in the below application root detection is implemented



Step2: To bypass root detection connect your device to Frida server with the below command



Step3: Run firda-ps service to view all the installed package on your device.

PID	Name	Identifier
3623	Allsafe	infosecadventures.allsafe
2817	Chrome	com.android.chrome
3546	Gallery	com.android.gallery3d
1381	Google Play Store	com.android.vending
1587	MEmu Launcher2	com.microvirt.launcher2
-	DCB Bank	com.dcb
-	File Manager	com.cyanogenmod.filemanager
-	Files	com.android.documentsui
-	Google Apps	com.microvirt.installer
-	Google Play Games	com.google.android.play.games
-	MEmu Guide	com.microvirt.guide
-	Settings	com.android.settings

Step4: Go on Codeshare. Frida page for root bypass script.

← → ↻

codeshare.frida.re

with-frida/

PROJECT PAGE

fridantiroot

👍 42 | 👁 145K

Uploaded by: @dzonerzy

Android antiroot checks bypass

PROJECT PAGE

0

👍

Uj

Ol

or

ot

an

sta

Step5: Copy the command to use the following Frida script.

← → ↻

codeshare.frida.re/@dzonerzy/fridantiroot/

☆

Frida CodeShare

Project: fridantiroot

Try this code out now by running

\$ frida --codeshare dzonerzy/fridantiroot -f YOUR_BINARY

```

1 //
2 Original author: Daniele Linguaglossa
3 28/07/2021 Edited by Simone Quadrini
4 Code amended to correctly run on the latest frida version
5 Added controls to exclude Magisk Manager
6 ~//
7
8 Java.perform(function() {
9   var RootPackages = ["com.noshufou.android.su", "com.noshufou.android.su.elite", "eu.chainfire.supersu",
10    "com.koushikdutta.superuser", "com.thirdparty.superuser", "com.yellowes.su", "com.koushikdutta.rommanager",
11    "com.koushikdutta.rommanager.license", "com.dimonvideo.luckypatcher", "com.chelpus.luckypatch",
12    "com.ramandroid.appquarantine", "com.ramandroid.appquarantinepro", "com.devadvance.rootcloak", "com.devadvance.rootcloakplus",
13    "de.robv.android.xposed.installer", "com.saurik.substrate", "com.zachspng.temprootremovejb", "com.amphoras.hidemyroot",
14    "com.amphoras.hidemyrootadfree", "com.for4m.hiderootPremium", "com.for4m.hideroot", "me.phh.superuser",
15    "eu.chainfire.supersu.pro", "com.kingouser.com", "com.topjohnwu.magisk"
16  ];
17
18   var RootBinaries = ["su", "busybox", "supersu", "Superuser.apk", "KingoUser.apk", "SuperSu.apk", "magisk"];
19
20   var RootProperties = {
21     "ro.build.selinux": "1",
22     "ro.debuggable": "0",
23     "service.adb.root": "0",
24     "ro.secure": "1"
25   };
26

```

Step6: Paste the Frida script and run with your package name in command prompt.

```
>frida -U -l rootandsslbypass.js -f infosecadventures.allsaf

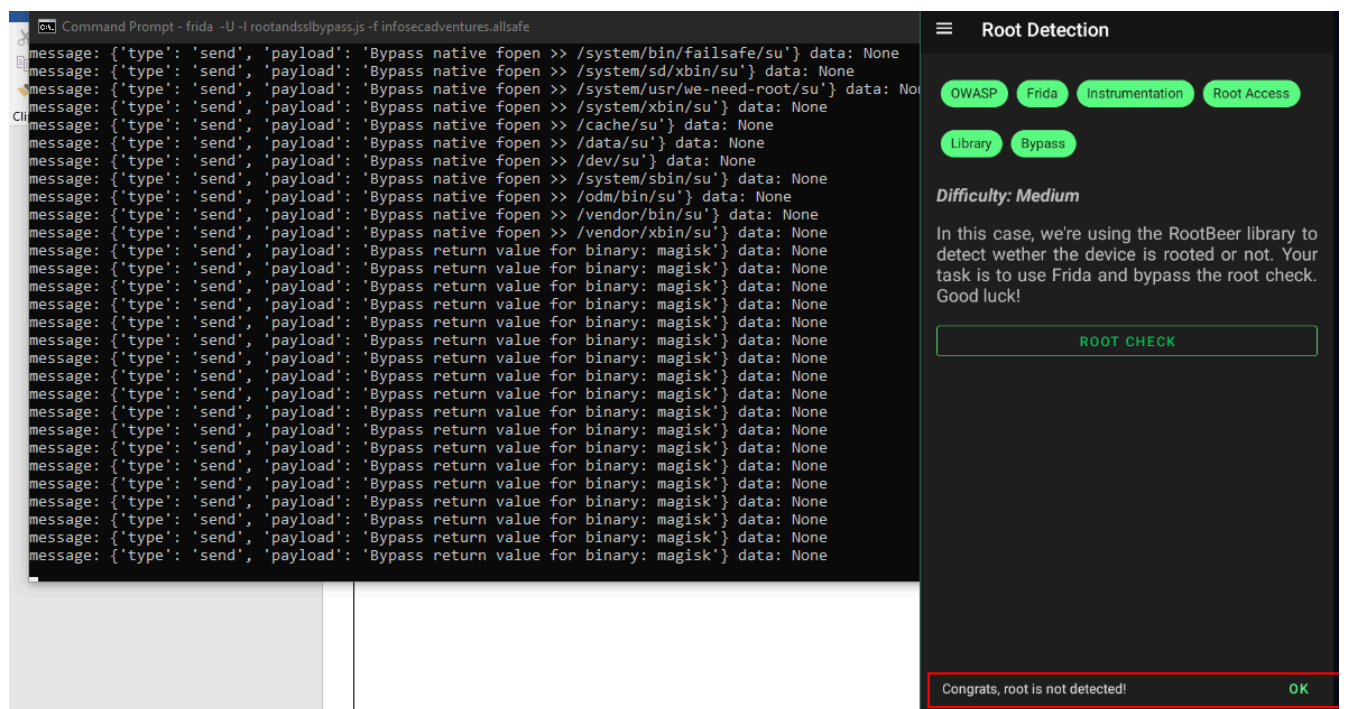
Frida 16.5.9 - A world-class dynamic instrumentation toolkit

Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit

More info at https://frida.re/docs/home/

Connected to ASUS Z01QD (id=127.0.0.1:21503)
Spawned `infosecadventures.allsafe`. Resuming main thread!
[ASUS Z01QD::infosecadventures.allsafe ]-> message: {'type': 'send', 'payload': 'Loaded 10652 classes!'} data: None
message: {'type': 'send', 'payload': 'loaded: -1'} data: None
message: {'type': 'send', 'payload': 'ProcessManager hook not loaded'} data: None
Horangi - Bypass Flutter SSL Pinning
```

Step7:Observer that the root detection is successfully bypass



Activity bypass using objection

Activity bypass specifically may involve circumventing an app's restrictions or detection mechanisms tied to particular activities (like login, authentication, or critical business logic) within the application. Objection allows you to interact with the activity manager, modify activity states, or even hide certain activities that would otherwise trigger security checks.

Step1: Use the following command to connect your application with objection

```
>objection -g com.mwr.example.sieve explore  
Using USB device `ASUS Z01QD`  
Agent injected and responds ok!  
  
[.] [.] [-] [-] [.]  
|_| |_| |_| |_| |_| |_  
    |(object)inject(ion) v1.11.0
```

Runtime Mobile Exploration
by: @leonjza from @sensepost

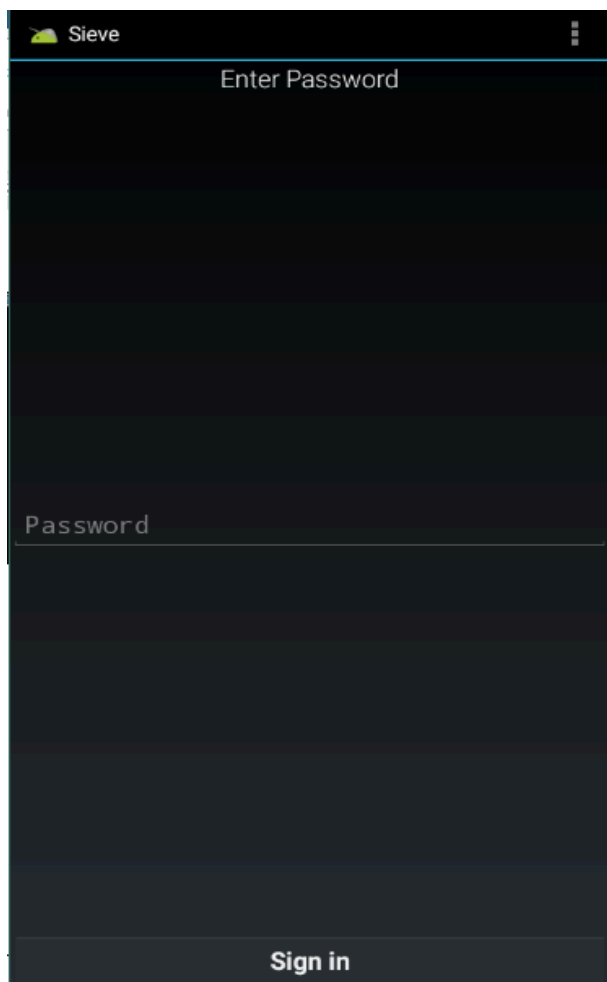
[tab] for command suggestions

Step2: Use hooking list to list all the activities of the application

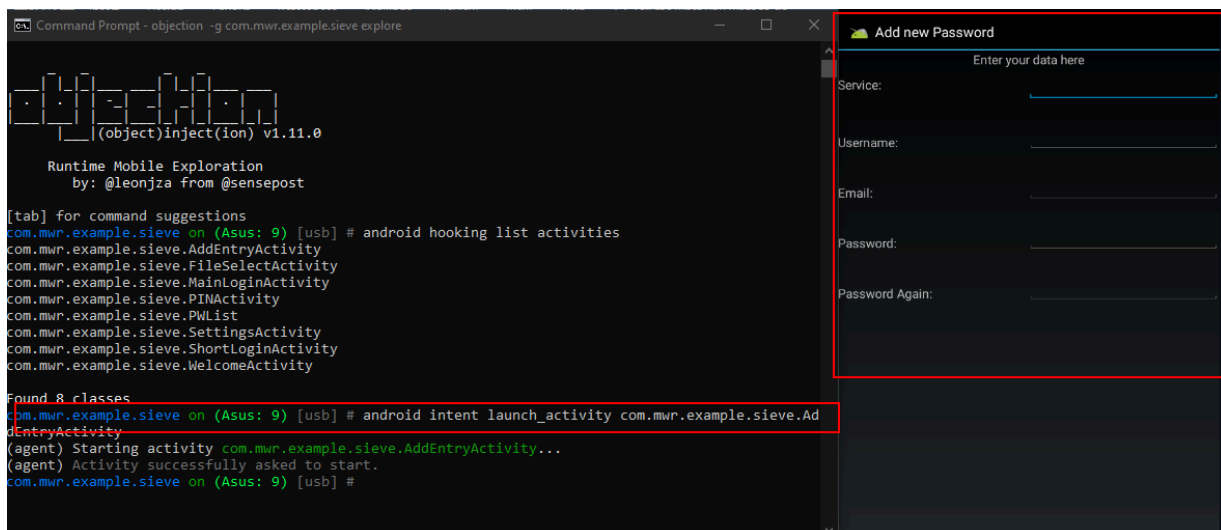
```
com.mwr.example.sieve on (Asus: 9) [usb] # android hooking list activities
com.mwr.example.sieve.AddEntryActivity
com.mwr.example.sieve.FileSelectActivity
com.mwr.example.sieve.MainLoginActivity
com.mwr.example.sieve.PINActivity
com.mwr.example.sieve.PWList
com.mwr.example.sieve.SettingsActivity
com.mwr.example.sieve.ShortLoginActivity
com.mwr.example.sieve.WelcomeActivity

Found 8 classes
com.mwr.example.sieve on (Asus: 9) [usb] #
```

Step3: The following application has a feature to add entries which is accessible after login .

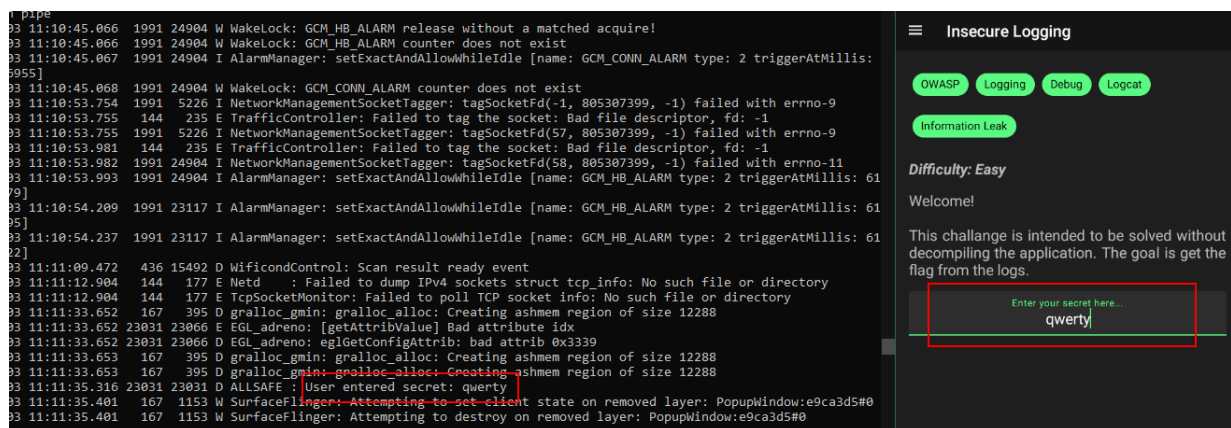
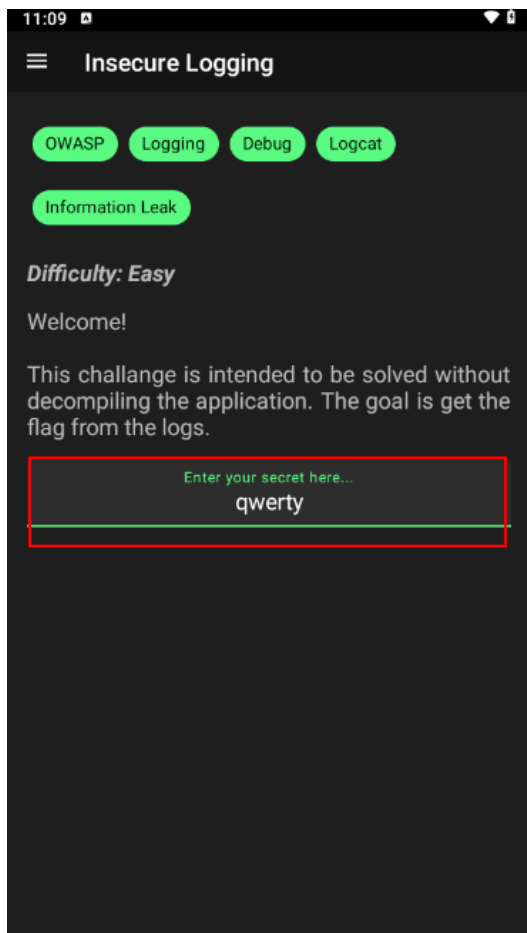


Step4: Using the following command for objection the following activity can be open without authentication .



ADB Logcat

ADB logcat is a command used in the Android Debug Bridge (ADB) to view and collect logs from an Android device or emulator. It allows developers and security researchers to monitor the system, application logs, and debug information in real-time, which is crucial for troubleshooting, debugging, and analyzing Android apps.



8. Dynamic Analysis with Burp